

DPLEvolve: Artifact Evaluation

Paper: "From Tool Invocation to Source-Mechanism Exploration: Protected White-Box DSE for Open-Source EDA"

Artifact version: v1.0.0 | **DOI:** [TO BE FILLED]

Authors & Contact

[TO BE FILLED]

Project Lead: [TO BE FILLED] **Contact:** [TO BE FILLED]

Overview

Traditional Design Space Exploration treats the EDA tool as a black box: parameters go in, QoR numbers come out, and you never learn *why* one configuration outperforms another.

DPLEvolve takes a different approach. It reads OpenROAD's source code, instruments individual optimization passes, and traces the causal chain from each transformation to its PPA impact. The result is white-box DSE: you understand not only that "parameter X works better," but that it works better *because* it reduces wirelength in post-CTS buffering.

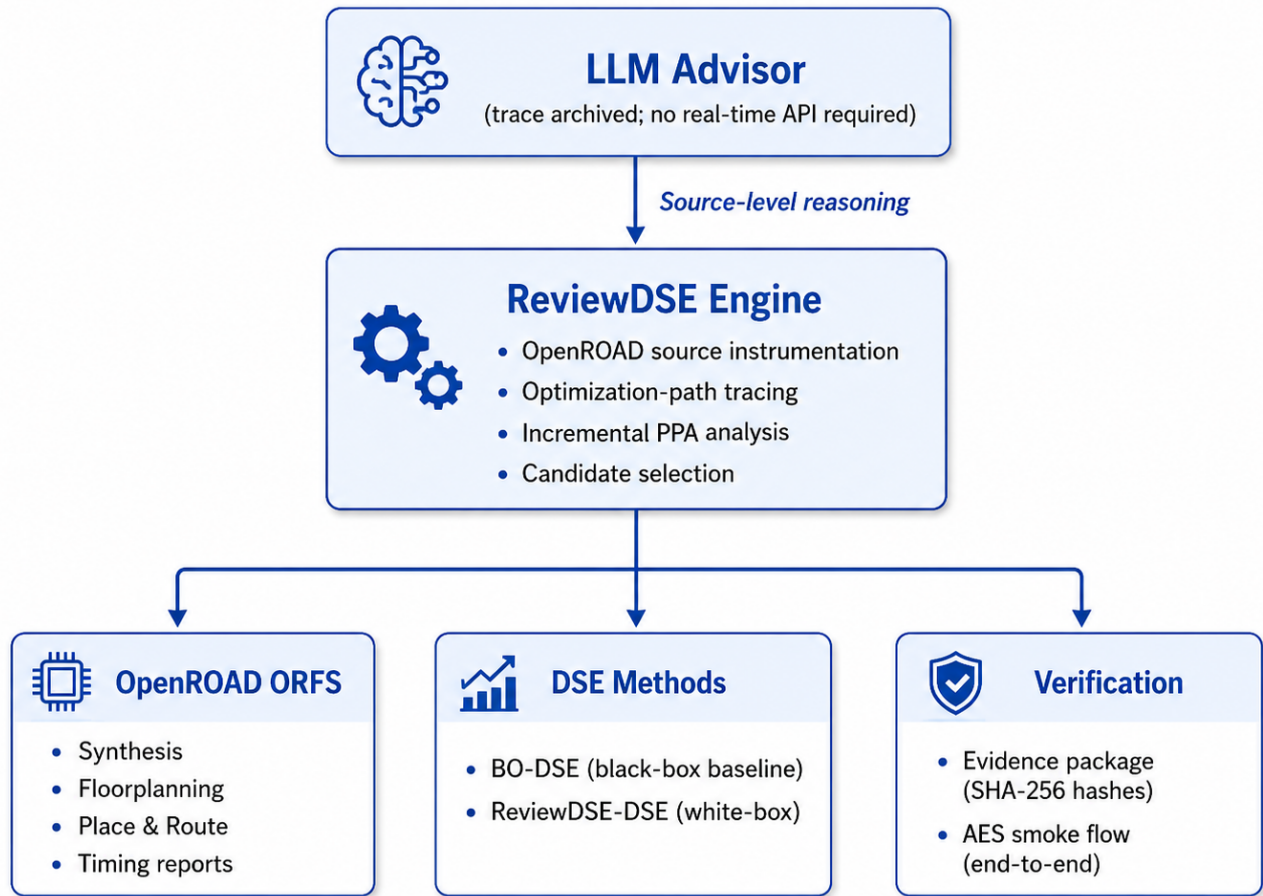
This artifact contains:

- **Source code** for the ReviewDSE framework
- **Pre-computed experiment traces** for all paper tables and figures
- **Web Demo** — a visual reviewer console for evidence checks, reproduction, live logs, and project slides
- **AES smoke flow** — a fresh RTL-to-GDS run you can execute on your own machine
- **SHA-256 integrity checks** on every evidence bundle

Packaged results can be verified in seconds. Running the full AES smoke flow from source takes approximately half an hour.

Architecture

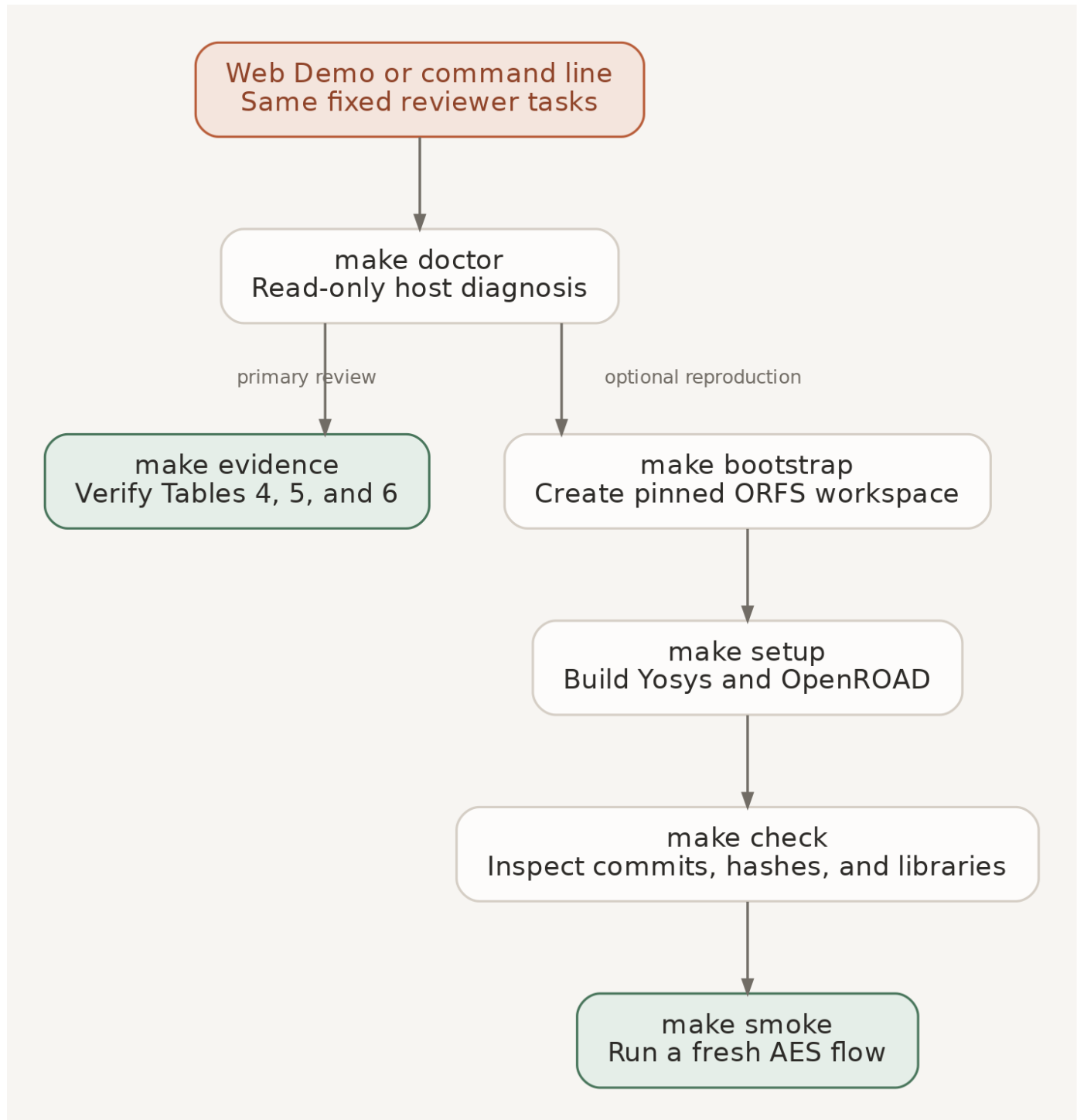
Overview of the ReviewDSE Artifact Workflow



The pipeline operates in six stages:

1. ORFS synthesizes and floorplans each design, producing ODB snapshots.
2. BO-DSE (baseline) searches the tool's exposed parameter space blindly.
3. ReviewDSE (our method) instruments OpenROAD source, traces each optimization's mechanism, and selects candidates with causal reasoning.
4. The LLM advisor attributes each QoR change to a specific code path. All analyses are archived as readable text — no API calls are needed during evaluation.
5. Evidence bundles cross-check every paper claim against archived expected values. No GPU or EDA license is required.
6. The AES smoke flow validates the complete toolchain from source to GDS.

Verification Flow



Step 1 — Reviewer Doctor (< 1 second): Checks Python, GNU Make, Bash, web-console support, build commands, memory, disk, repository inputs, ORFS, and EDA binaries. It never installs packages; missing items produce copyable, OS-specific commands for the reviewer to run manually.

Step 2 — Evidence verification (< 5 seconds): Cross-checks every pre-computed result against its expected value. This is the primary verification path — it confirms that the packaged evidence matches the paper's claims.

Step 3 — Prepared environment inspection (< 1 second, after setup): Checks pinned source revisions, Yosys/OpenROAD binaries, hashes, and shared libraries.

Step 4 — Smoke flow (~20–35 minutes including setup, optional): Builds Yosys and OpenROAD from

pinned source revisions, then runs synthesis → floorplan → place → route on the AES (Nangate45) design. Run this to verify that the entire toolchain functions end-to-end on your hardware.

Repository Structure

DPLEvolve-AE/	
— README.md	# Primary reviewer guide
— README.pdf	# PDF copy of the reviewer guide
— Makefile	# All command-line entry points
— dplevolve-architecture.png	# Architecture diagram
— dplevolve-verification-flow.png	# Verification flow diagram
— web-demo/	# Visual artifact reviewer console
— server.py	# FastAPI task queue and SSH backend
— start.sh	# Isolated web environment launcher
— requirements.txt	# Pinned-compatible web dependencies
— templates/	# English UI, guide, and project slides
— static/	# Local visual assets
— tests/	# Web backend regression tests
— artifacts/	# Experiment results and evidence
— 01-table4-qor/	# Table 4: QoR comparison
— expected/	# Authoritative expected values
— traces/	# Readable archived reasoning traces
— selected-programs/	# 18 source trees and SHA-256 manifests
— output/	# Generated verification output
— 02-table5-composability/	# Table 5: composability
— 03-table6-cutrow/	# Table 6: cut-row patterns
— 04-aes-smoke/	# Fresh and archived AES smoke checks
— src/dpl_evolve_agent/	# ReviewDSE implementation
— docs/	# Reviewer and technical documentation
— quickstart.md	# Detailed execution instructions
— environment.md	# Pinned tools and machine setup
— expected-results.md	# Expected values and tolerances
— troubleshooting.md	# Diagnosis and remediation guidance
— claims-to-artifacts.md	# Paper-claim-to-command mapping
— scripts/	# Human, agent, and shared scripts
— provenance/	# Pinned revisions and checksums
— source-commits.json	# Exact tool commit hashes
— tests/	# Structure, integration, and unit tests
— paper/	# Paper PDF and notes
— agent/	# Agent task definitions and schemas

Paper-to-Artifact Mapping

Each claim in the paper that this artifact supports is listed below. All seven pass make evidence on a clean checkout. Only C5 requires EDA tools; the remaining claims run on Python standard library alone.

ID / Paper	Claim	Command	Input	Expected Result / Time
C1 / Table 4	ReviewDSE-HPWL reduces wirelength by 1.78% vs BO-DSE baseline	make table4	Table 4 bundle	Table 4 QoR CSV; < 5 sec

ID / Paper	Claim	Command	Input	Expected Result / Time
C2 / Table 4	ReviewDSE-GHR reduces global route overflow by 1.68%, 1.11x runtime	make table4	Table 4 bundle	Table 4 GHR values; < 5 sec
C3 / Table 5	3 counterexamples demonstrate ReviewDSE composability	make table5	Table 5 bundle	3 matching verdicts; < 1 sec
C4 / Table 6	9 cut-row repair patterns verified across designs	make table6	Table 6 bundle	9 matching cases; < 1 sec
C5 / Sec. V-C	AES smoke flow: full RTL-to-GDS runs on Nangate45	make bootstrap && make setup && make smoke	10 GB; network during setup	[OK] AES smoke test PASSED; setup 10-30 min plus smoke 2-5 min
C6 / Table 4	18 selected program source trees for audit	make table4	Table 4 selected programs	18 trees with SHA-256 manifests; < 5 sec
C7 / Sec. IV	ReviewDSE source code	Source inspection	src/	Documented Python source

Hardware & Software Requirements

Hardware

- **OS:** Linux x86-64 (Ubuntu 20.04 or 22.04 tested)
- **CPU:** Any for evidence checks; 2+ cores (4 recommended) for smoke flow
- **GPU:** Not required
- **RAM:** < 1 GB for evidence; 8 GB (16 GB recommended) for smoke
- **Disk:** ~1 GB for evidence; ~10 GB for smoke
- **Network:** Required only during setup (cloning repositories, ~2 GB download)
- **Expected total runtime:** Seconds for evidence; ~10-30 minutes (setup) + ~2-5 minutes (smoke)

Software

Evidence verification:

- Python >= 3.11
- GNU Make >= 4.0
- Bash >= 4.0

No pip packages, EDA tools, or API keys are required for evidence checking.

Smoke flow (auto-installed):

- Yosys 0.41 (built from pinned commit in provenance/source-commits.json)
- OpenROAD v2.0 (built from pinned commit)
- All remaining dependencies are fetched and built by `make bootstrap && make setup`.

Environment Variables

Variable	Default	Description
DPL_EVOLVE_PYTHON	python3	Override Python interpreter path
SMOKE_THREADS	nproc	Thread count for smoke build
DPL_EVOLVE_THREADS	4	Thread count for agent-dispatched runs
ORFS_ROOT	../OpenROAD-flow-scripts	Override ORFS workspace path
DPL_EVOLVE_STATE_ROOT	../dpl_evolve_state	State directory (auto-detected)
DPL_EVOLVE_AGENT_ROOT	src/dpl_evolve_agent	Agent root (auto-detected)

Web Demo (Recommended)

The artifact includes an English visual reviewer console under `web-demo/`. It moves the fixed command-line workflow into a browser while preserving the same Make targets, working directory, live output, and exit codes. It also includes a project Slides view covering the proposed DPLevolve architecture, the AE repository architecture, the verification workflow, and the reported DSE effects.

Using the Web Demo is sufficient for review: it can replace both the command-line **Quick Start** and **Full Reproduction** sections below. Those sections remain as equivalent references for reviewers who prefer a terminal or need automation.

Start the console from the repository root:

```
bash web-demo/start.sh
```

Then open:

- Review console: `http://127.0.0.1:8080/#review`
- Project slides: `http://127.0.0.1:8080/#slides`
- Reviewer guide: `http://127.0.0.1:8080/#guide`

The recommended browser workflow is:

1. Run **make doctor** and follow any Suggested commands in a normal terminal.
2. Run **make evidence**, or inspect Tables 4–6 individually.
3. Optionally run **make doctor-smoke**, **bootstrap**, **setup**, **check**, and **smoke**; the **full reproduction** button runs the preparation and fresh smoke path in sequence.
4. Read the live terminal output and confirm a zero exit code in Run history.

If the repository is on a remote Linux server, keep the console bound to loopback and create a tunnel from the reviewer's computer:

```
ssh -L 8080:127.0.0.1:8080 USER@SERVER
```

Then open the same `http://127.0.0.1:8080/` address locally. The launcher creates an isolated `web-demo/.venv` and installs only the web application's Python dependencies. It does not install system packages or EDA tools. Because the console can launch builds, do not expose it directly to the public Internet. See `web-demo/README.md` for connection modes and troubleshooting.

Quick Start (Command Line)

```
make doctor          # Read-only diagnosis; prints exact commands for missing prerequisites
make evidence        # Verify all packaged results against expected values
```

Doctor works before ORFS exists and never installs packages. Review and run any suggested remediation command manually, then repeat Doctor. If both commands pass, the packaged artifact evidence is verified. `make evidence` should produce:

```
[PASS] All packaged paper-evidence bundles passed
```

Individual tables can also be verified:

```
make table4          # Table 4: QoR comparison
make table5          # Table 5: Composability
make table6          # Table 6: Cut-row patterns
```

Full Reproduction: Smoke Flow (Command Line)

The smoke flow rebuilds Yosys and OpenROAD from source and runs a fresh AES design through synthesis, floorplanning, placement, and routing.

```

make doctor-smoke    # Strictly check full-reproduction readiness
make bootstrap       # Clone pinned Yosys and OpenROAD revisions (~2 min)
make setup           # Build from source (~10–30 min, one-time)
make check           # Inspect prepared commits, binaries, hashes, and libraries
make smoke           # Run AES RTL-to-GDS (~2–5 min after build)

```

Expected output: [OK] AES smoke test PASSED

- bootstrap clones the exact commits recorded in provenance/source-commits.json.
- setup compiles both tools from those commits. This takes 10–30 minutes on a typical machine.
- smoke runs the full flow on AES (Nangate45) and compares the output against the lock file at artifacts/04-aes-smoke/expected/ae_reproduction_lock.json.

To verify archived smoke results without re-running:

```
make smoke-check
```

Output Locations

Command	Output
make evidence	Read-only check; no output files
make table4/5/6	artifacts/0X-*/output/
make smoke	artifacts/04-aes-smoke/output/
make clean	Deletes outputs; preserves evidence

Expected Results

The authoritative reference is the JSON files under artifacts/*/expected/. Approximate values are provided below for orientation.

Bundle	Approximate Value	Reference File
BO-DSE	~0.38% mean HPWL reduction	artifacts/01-table4-qor/expected/table4.json
ReviewDSE-HPWL	~1.78% reduction, ~1.34× runtime	artifacts/01-table4-qor/expected/paper_claims.j
ReviewDSE-GHR	~1.68% reduction, ~1.11× runtime	artifacts/01-table4-qor/expected/paper_claims.j
AES smoke	Exact values in lock file	artifacts/04-aes-smoke/expected/ae_reproduction

Small deviations within the stated tolerances are normal.

Reproducibility Notes

Included

All results in Tables 4–6 are backed by archived data with SHA-256 integrity checks. The AES smoke flow is fully reproducible from source.

Not Included (and why)

ODB input files (~3 TB). The nine design databases from the paper's experimental runs are too large to distribute. Exact OpenROAD and Yosys commit hashes are documented in provenance/source-commits.json, along with RTL sources. Rebuilding from the same commits produces functionally identical ODBs.

Full LLM trace re-generation (~2B tokens per design). The ReviewDSE traces in Table 4 were generated via a proprietary LLM API. Re-running from scratch costs approximately \$30–50 per design

and requires live API access. All 18 candidate traces are archived as readable text in `artifacts/01-table4-qor/traces/` and validated by SHA-256 checks. The reasoning can be audited without API calls.

Per-run EDA logs for Tables 5/6. Compact summaries are provided instead. Provenance hashes are recorded in each bundle's `inputs/provenance.json`.

Troubleshooting

Problem	Solution
python3: command not found	Install Python 3.11+, or set <code>DPL_EVOLVE_PYTHON=/path/to/python3</code>
make: command not found	<code>apt-get install build-essential</code> (Ubuntu)
Permission denied on scripts	<code>bash artifacts/.../run.sh</code> (executable bit not required)
Evidence digest mismatch	Do not edit files under <code>expected/</code> . Run <code>git status</code> to inspect changes.
make check fails	See <code>docs/environment.md</code> for required tool versions.
Smoke cannot find ORFS	Run <code>make bootstrap</code> first, or set <code>ORFS_ROOT</code> to your ORFS checkout.
Smoke hash mismatch	Verify commits in <code>provenance/source-commits.json</code> match your checkout.
OOM during smoke	Reduce threads: <code>SMOKE_THREADS=2 make smoke</code>
Smoke build is slow	Normal. Compiling Yosys and OpenROAD from source takes 10-30 minutes.

Further detail: `docs/troubleshooting.md`

Current Limitations

- **Research framework.** DPLevolve is academic work on DSE methodology. It does not provide signoff-quality timing closure.
- **Pre-computed LLM traces.** Traces can be read and audited, but generating new ones requires proprietary API access.
- **Linux x86-64 only for smoke flow.** Evidence checks run on any operating system with Python 3.11+. The smoke flow requires Linux.
- **ODB regeneration requires compilation.** Functionally identical ODBs can be rebuilt from pinned commits, but a full compile cycle is necessary.

License & Citation

BSD 3-Clause. See [LICENSE](#). Machine-readable: [CITATION.cff](#)

```
@inproceedings{dplevolve2026,  
  title      = {From Tool Invocation to Source-Mechanism Exploration:  
               Protected White-Box DSE for Open-Source EDA},  
  author     = {[TO BE FILLED]},  
  booktitle  = {MLCAD},  
  year       = {2026}  
}
```

Questions: [GitHub Issues](#) or email [TO BE FILLED]